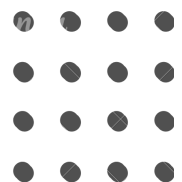




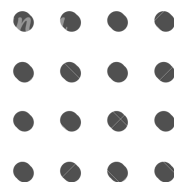
VMES Grinds





Pilot Program - Week 4 - Day 3

Heap





3266. Final Array State After K Multiplication Operations II





3266. Final Array State After K Multiplication Operations II

Hard

Topics

Companies

Hint

You are given an integer array `nums`, an integer `k`, and an integer `multiplier`.

You need to perform `k` operations on `nums`. In each operation:

- Find the **minimum** value `x` in `nums`. If there are multiple occurrences of the minimum value, select the one that appears **first**.
- Replace the selected minimum value `x` with `x * multiplier`.

After the `k` operations, apply **modulo** `$10^9 + 7$` to every value in `nums`.

Return an integer array denoting the *final state* of `nums` after performing all `k` operations and then applying the modulo.

Constraints:

- `1 <= nums.length <= 104`
- `1 <= nums[i] <= 109`
- `1 <= k <= 109`
- `1 <= multiplier <= 106`

**Example 1:**

Input: `nums = [2,1,3,5,6], k = 5, multiplier = 2`

Output: `[8,4,6,5,6]`

Explanation:

Operation	Result
After operation 1	[2, 2, 3, 5, 6]
After operation 2	[4, 2, 3, 5, 6]
After operation 3	[4, 4, 3, 5, 6]
After operation 4	[4, 4, 6, 5, 6]
After operation 5	[8, 4, 6, 5, 6]
After applying modulo	[8, 4, 6, 5, 6]

Example 2:

Input: `nums = [100000,2000], k = 2, multiplier = 1000000`

Output: `[999999307,999999993]`

Explanation:

Operation	Result
After operation 1	[100000, 2000000000]
After operation 2	[100000000000, 2000000000]
After applying modulo	[999999307, 999999993]



Explanation





Thought Process

- **Linear Iteration** → to perform K operations
- **Use minHeap** → to find minimum value
- **In the end, perform modulo operation**
- **Problem is position of elements must be fixed.**
 - **So, index must be used in a way.**



Code





```
import heapq
class Solution:
    def getFinalState(self, nums: list[int], k: int, multiplier: int) -> list[int]:
        n = len(nums)
        minH = []
        res = [0] * n

        for i, val in enumerate(nums):
            heapq.heappush(minH, (val, i))

        for i in range(k):
            x, i = heapq.heappop(minH)
            newX = x * multiplier
            heapq.heappush(minH, (newX, i))

        for j in range(n):
            x, i = heapq.heappop(minH)
            res[i] = x % (10**9+7)

        return res
```



Time and Space Complexity Analysis





```
import heapq
class Solution:
    def getFinalState(self, nums: list[int], k: int, multiplier: int) -> list[int]:
        n = len(nums)
        minH = []
        res = [0] * n ← Space - O(n)

        for i, val in enumerate(nums): ← Time - O(n)
            heapq.heappush(minH, (val, i)) ← Time - O(log n)

        for i in range(k): ← Time - O(k)
            x, i = heapq.heappop(minH)
            newX = x * multiplier
            heapq.heappush(minH, (newX, i)) ← Time - O(log n)

        for j in range(n): ← Time - O(n)
            x, i = heapq.heappop(minH) ← Time - O(log n)
            res[i] = x % (10**9+7)

        return res
```

Overall: Time - $O(n \log n + k \log n) \rightarrow O(k \log n)$
Because [k can be as large as 10^9]



Optimization





Time Limit Exceeded (TLE) Problem

- **Cases**
 - whenever K is LARGE ($\geq 10^6$).
 - when values get LARGE
- **Worst-case scenario**
 - suppose Heap operations: $\log n \approx 14$
 - $k * \log n = 1,000,000,000 \times 14$
 - 14,000,000,000 operations



How to Handle this?

- Problem has two distinct phases.
- The theory involves "Greedy Catch-up" followed by "Modular Exponentiation".

So,

- Avoid simulating each operation.
- Group multiplications using exponentiation.
- Stop early once values exceed original max.



Phase 1: The "Catch-up" Phase (**Heap Simulation**)

- **At the start, array values differ greatly** — some small, some large.
- Min-heap **repeatedly picks smallest value and multiplies** → "catch up" to the larger values.
- Why simulation is needed?
 - Smallest element is still far behind the current maximum, so **multiplying changes the heap order**.
- This phase continues until
 - **All operations ($k == 0$)**, or
 - Smallest value, after multiplication, would become larger than (or comparable to) the current max → the array becomes **"stable."**



Phase 2: “Steady State” Phase (Math, Not Simulation)

- Array becomes **balanced**.
 - All values are now similar in size.
 - Multiplying the smallest **no longer changes the overall order**.
- Behavior becomes **cyclic**.
 - The “minimum” rotates through the array in a fixed cycle.
 - **index A → index B → index C → ... → index Z → index A → ...**
- **Math replaces simulation**.
- Suppose N elements and k operations left.
 - Full Cycles → Each element gets multiplied **$k // N$** times.
 - Remaining Steps → Smallest **$k \% N$** elements receive one extra multiplication.
- **No more heap operations needed** — everything is **predictable**.



Example: nums = [2, 10, 20] (N = 3), multiplier = 2, k = 11

Step 1 → [4, 10, 20]

Step 2 → [8, 10, 20]

Step 3 → [16, 10, 20]

Step 4 → [16, 20, 20]

Step 5 → [32, 20, 20]

Step 6 → [32, 40, 20]

Step 7 → [32, 40, 40]

Step 8 → [64, 40, 40]

Step 9 → [64, 80, 40]

Step 10 → [64, 80, 80]

Step 11 → [128, 80, 80]

=====

Step 1 to 4 = Phase 1

[16, 20, 20] = balanced

=====

Step 5 to 11 = Phase 2

Cyclic behavior

k = 11 - 4 = 7

(because 4 operations performed in Phase 1)

k // N = 7 // 3 = 2 cycles (step 5 to 7 & step 8 to 10)

k % N = 7 % 3 = 1 extra (step 11)

=====



Optimized Code





```
import heapq
class Solution:
    def getFinalState(self, nums: list[int], k: int, multiplier: int) -> list[int]:
        if multiplier == 1:
            return nums

        n = len(nums)
        maxVal = max(nums)

        minH = [(x, i) for i, x in enumerate(nums)]
        heapq.heapify(minH)

        while 0 < k:
            x, i = minH[0]
            newX = x * multiplier
            if newX > maxVal:
                break
            heapq.heappop(minH)
            heapq.heappush(minH, (newX, i))
            k -= 1

        exp = k // n
        rem = k % n

        m = 10**9+7

        extra = pow(multiplier, exp+1, m)
        normal = pow(multiplier, exp, m)

        res = [0] * n

        for j in range(n):
            x, i = heapq.heappop(minH)
            if j < rem:
                res[i] = (x % m * extra) % m
            else:
                res[i] = (x % m * normal) % m

        return res
```



Time and Space Complexity Analysis





```
import heapq
class Solution:
    def getFinalState(self, nums: list[int], k: int, multiplier: int) -> list[int]:
        if multiplier == 1:
            return nums

        n = len(nums)
        maxVal = max(nums)

        minH = [(x, i) for i, x in enumerate(nums)]
        heapq.heapify(minH)

        while 0 < k:
            x, i = minH[0]
            newX = x * multiplier
            if newX > maxVal:
                break
            heapq.heappop(minH)
            heapq.heappush(minH, (newX, i))
            k -= 1

        exp = k // n
        rem = k % n

        m = 10**9+7

        extra = pow(multiplier, exp+1, m)
        normal = pow(multiplier, exp, m)

        res = [0] * n

        for j in range(n):
            x, i = heapq.heappop(minH)
            if j < rem:
                res[i] = (x % m * extra) % m
            else:
                res[i] = (x % m * normal) % m

        return res
```

← Time - $O(n)$

← Time - $O(t)$ [t means early break]

← Time - $O(\log n)$

← Space - $O(n)$

← Time - $O(n)$

← Time - $O(\log n)$

Overall: Time - $O(n \log n + t \log n) \rightarrow O(n \log n)$



Key Takeaway





- **Naive heap simulation is impossible.**
- **Optimize solution with two phases.**
- **Instead of simulating all operations, detect repetition and switch to a mathematical formula.**
- **Avoids unnecessary operations and prevents timeouts.**



Thank You !